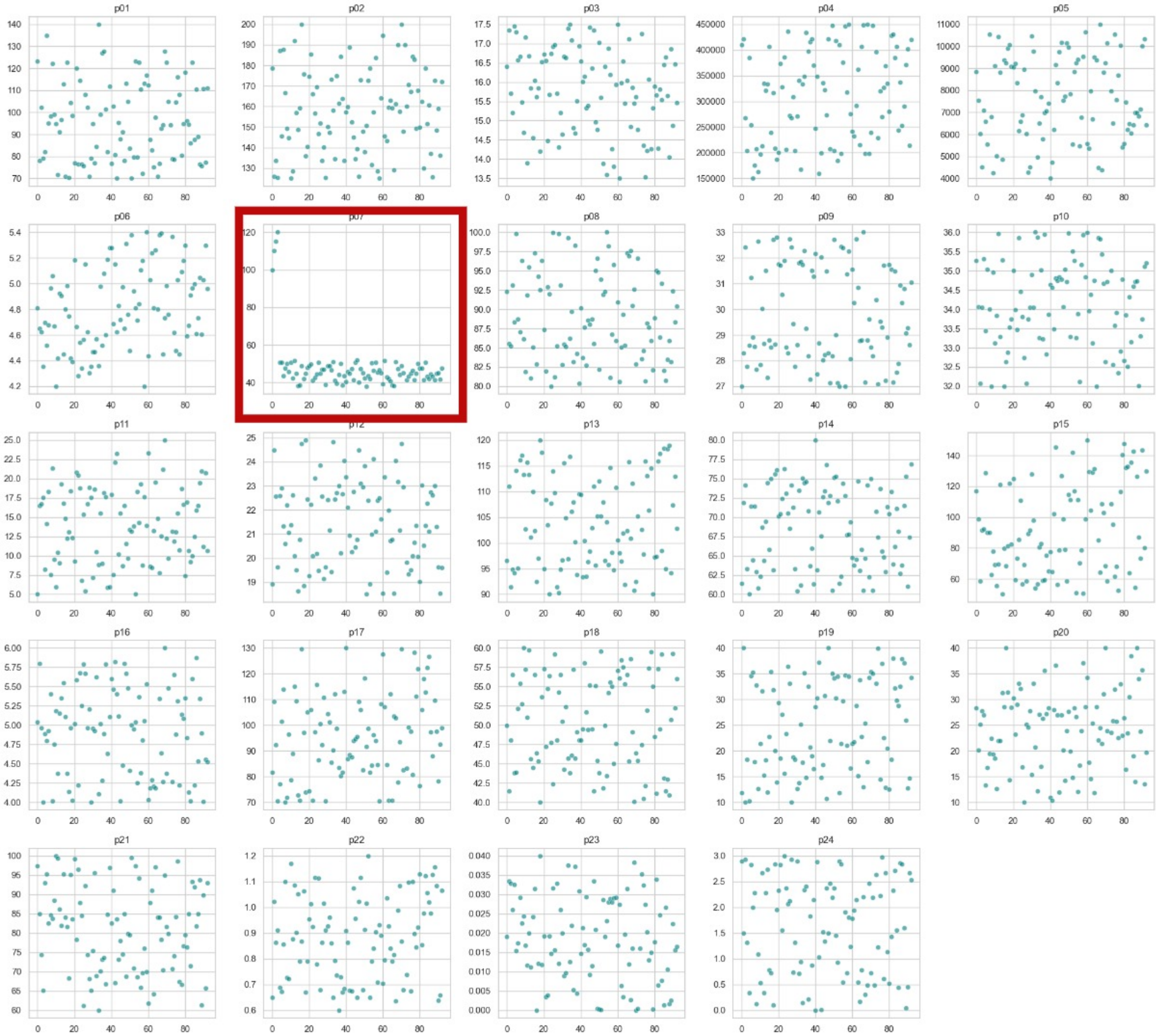
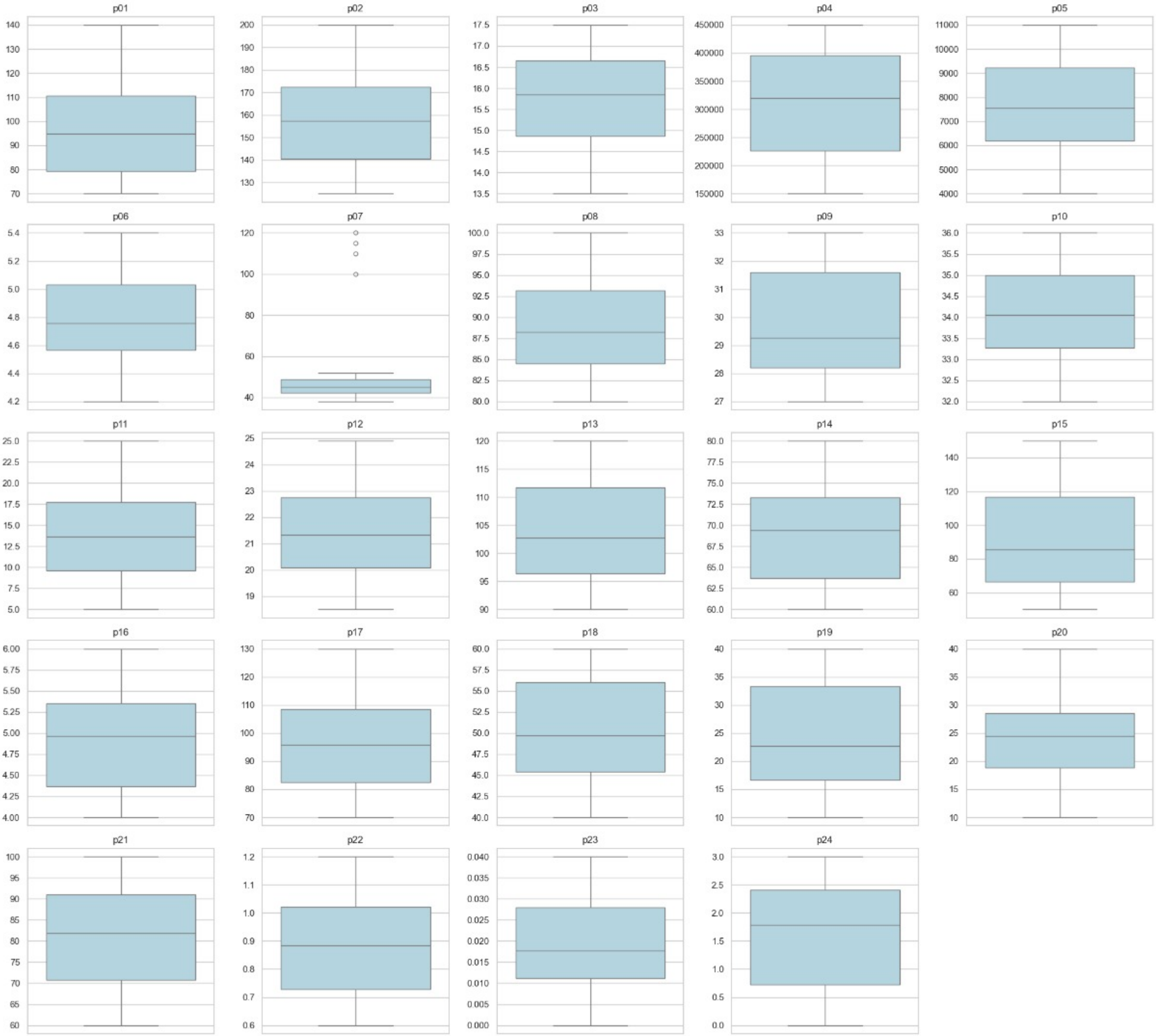


Global Analysis: Scatter Plot (vs Index)



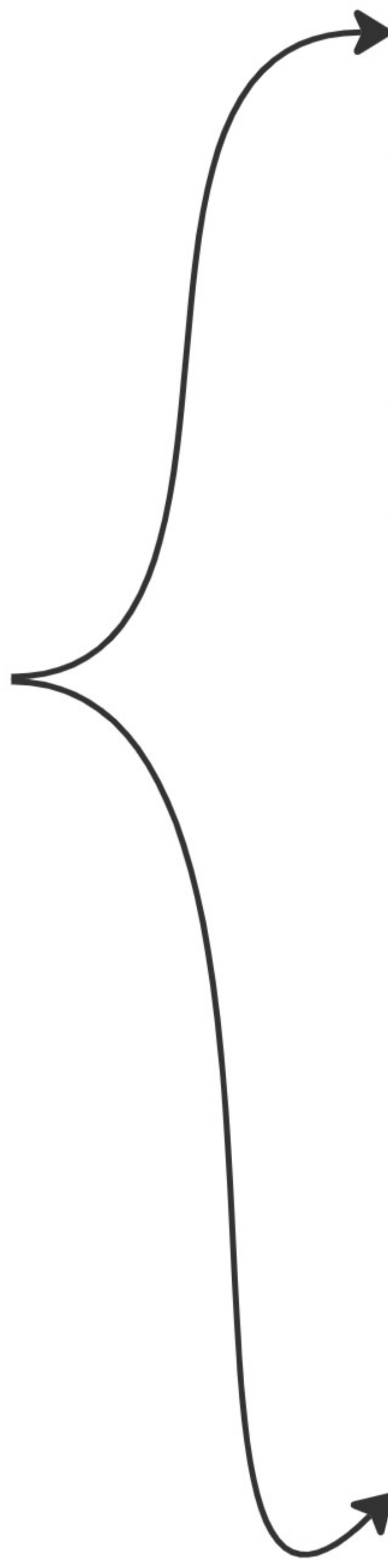
Global Analysis: Box Plot



Following is a list of common tasks involved in data cleaning:

1. **Error correction:** Identifying and correcting errors in the data, such as misspellings, typographical errors, or incorrect values
2. **Duplicate removal:** Identifying and removing duplicate entries in the data set to avoid redundancy and ensure data integrity
3. **Missing data handling:** Dealing with missing data by either **imputing** values or deciding how to handle the missing data in the analysis
4. **Standardization (Data Scaling):** Ensuring consistency in the format, units, and representation of data across the data set
5. **Normalization (Data Scaling):** A data transformation technique that scales values within a range (often 0 to 1) to maintain relative relationships between features.
6. **Outlier detection and treatment:** Identifying outliers, which are extreme values that deviate significantly from the rest of the data, and deciding how to handle them, such as removing them or replacing them with appropriate values

Data cleaning is a crucial step in the data analysis process as it helps to ensure the accuracy and reliability of the results obtained from the data. It requires careful attention to detail and an understanding of the specific data set and its characteristics. By performing data cleaning, data practitioners can improve the quality of the data and make more informed decisions based on reliable and trustworthy information.

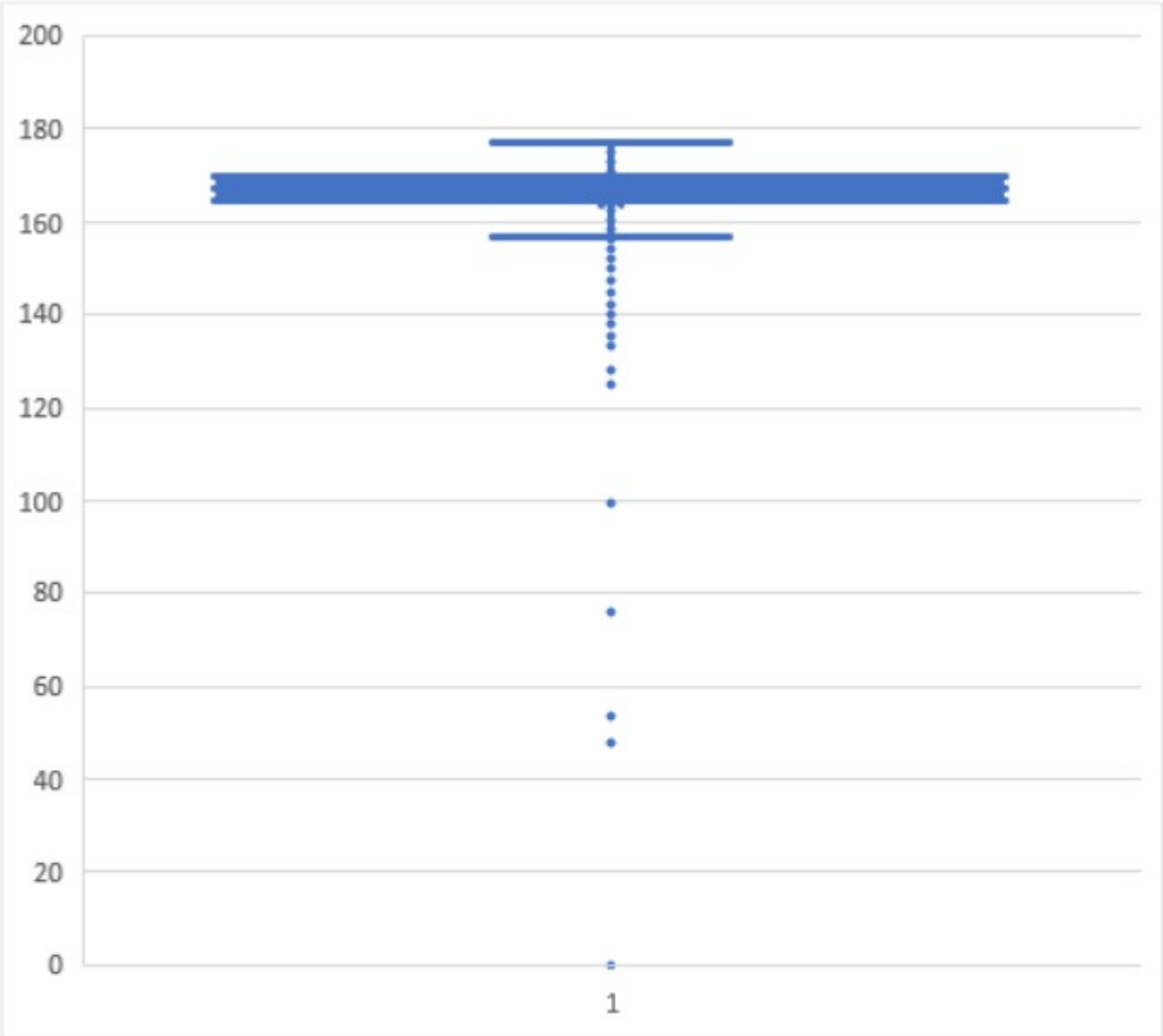


Column	Min	Max	Mean
c3	0	187.0776	170.3037
c4	0	174.0258	163.9285
c5	0	0.915783	0.529769
c6	0	3.144694	1.5678
c7	0.447406	2.971087	2.426871
c8	0	22.69527	21.20686
c9	0	10.27313	7.427598
c10	0	0.785255	0.642268
c11	0.002404	59.08272	56.35317
c12	0	13.7972	11.64599
c13	0	36.43647	32.86705
c14	0	5.409382	0.342937
c15	0	5.36033	1.531901
c16	0.019261	21.16151	17.79199
c17	0	50.36344	28.20165
c18	0	41.90749	30.87445
c19	0	17.54499	12.88846
c20	0	11.33466	6.722164
c21	-0.94329	11.36428	7.921272
c22	0	7.757059	4.729896
c23	14.98597	48.65412	42.24162
c24	0	0.915783	0.529769
c25	0	3.144694	1.5678

	A	B	C	D	E	F
6	c1	c2	c3	c4	c5	c6
7	01-Sep-13 00:00:00	1	161.1432	153.8311	0.63172	2.414032
8	02-Sep-13 00:00:00	1	164.1032	156.9562	0.656179	2.355487
9	03-Sep-13 00:00:00	1	165.3228	157.8616	0.639334	2.701887
10	04-Sep-13 00:00:00	1	165.7317	158.027	0.652345	2.459543
11	05-Sep-13 00:00:00	1	162.6875	155.1138	0.657465	1.909607
12	06-Sep-13 00:00:00	1	162.6016	154.7801	0.668482	1.52945
13	07-Sep-13 00:00:00	1	162.4659	154.8351	0.740854	1.409602
14	08-Sep-13 00:00:00	1	166.0759	158.1119	0.782362	1.449534
15	09-Sep-13 00:00:00	1	165.2525	157.0692	0.746412	1.515374
16	10-Sep-13 00:00:00	1	165.241	157.0487	0.745851	1.655467
17	11-Sep-13 00:00:00	1	165.4747	157.125	0.792748	1.573377
18	12-Sep-13 00:00:00	1	165.2264	157.442	0.788428	1.736178
19	13-Sep-13 00:00:00	1	165.9279	158.3007	0.730994	1.798794
20	14-Sep-13 00:00:00	1	168.2197	160.6276	0.618005	2.271551
21	15-Sep-13 00:00:00	1	166.0266	158.6136	0.574357	2.471939
22	16-Sep-13 00:00:00	1	166.8304	159.5535	0.603332	2.374928
23	17-Sep-13 00:00:00	1	164.1426	156.4439	0.592301	2.123248
24	18-Sep-13 00:00:00	1	166.4962	158.1226	0.612283	2.423496
25	19-Sep-13 00:00:00	1	162.8119	154.3308	0.574474	2.519138
26	20-Sep-13 00:00:00	1	160.8477	152.1846	0.386043	2.757094
27	21-Sep-13 00:00:00	1	155.8484	148.1267	0.39832	2.789573
28	22-Sep-13 00:00:00	1	148.2437	141.4792	0.415291	2.766708
29	23-Sep-13 00:00:00	1	147.8628	140.7158	0.377464	2.763958
30	24-Sep-13 00:00:00	1	151.7811	143.3661	0.473188	2.586392
31	25-Sep-13 00:00:00	1	154.0191	145.4945	0.507609	2.561311
32	26-Sep-13 00:00:00	1	153.9183	145.2496	0.508395	2.085402

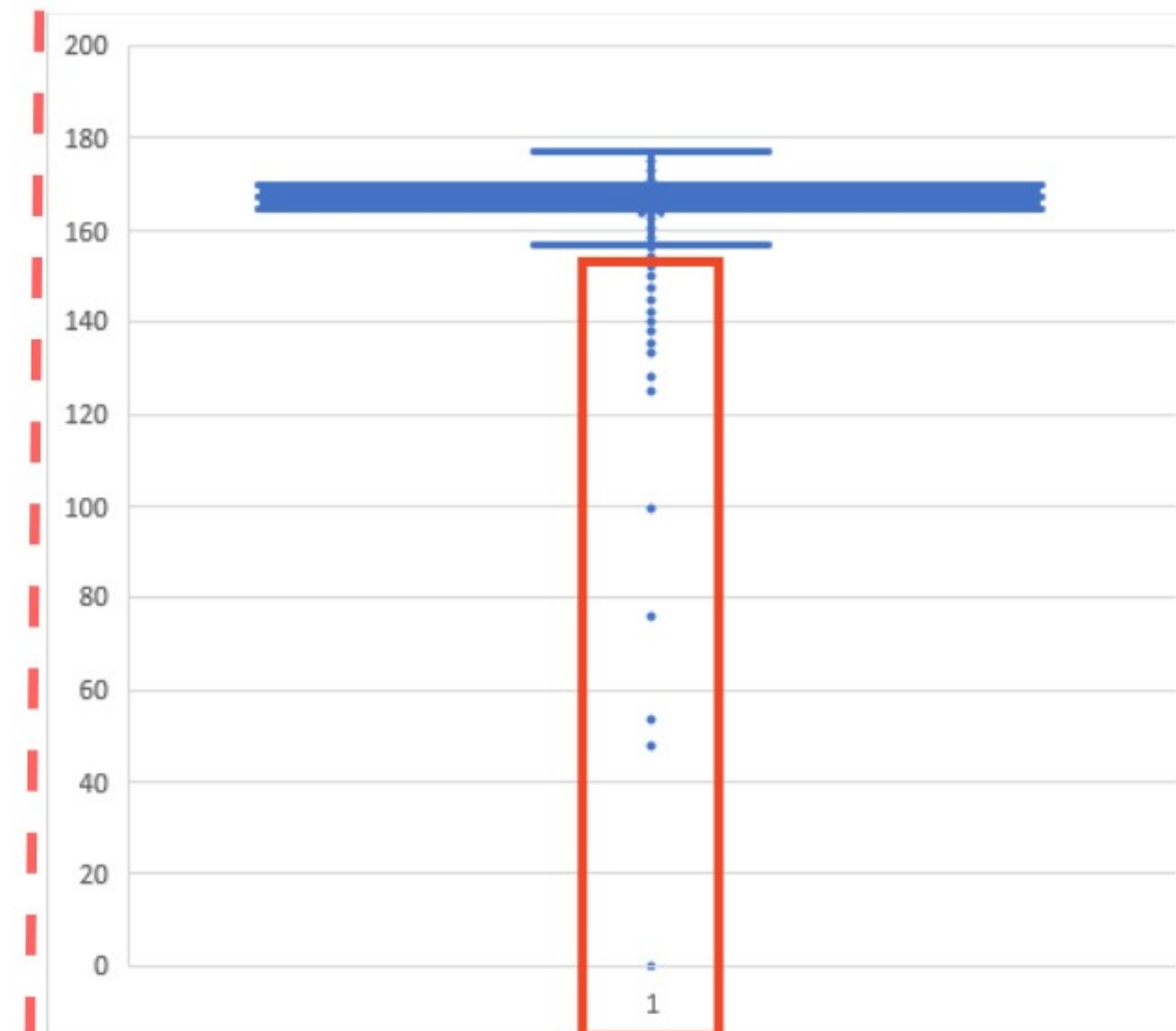
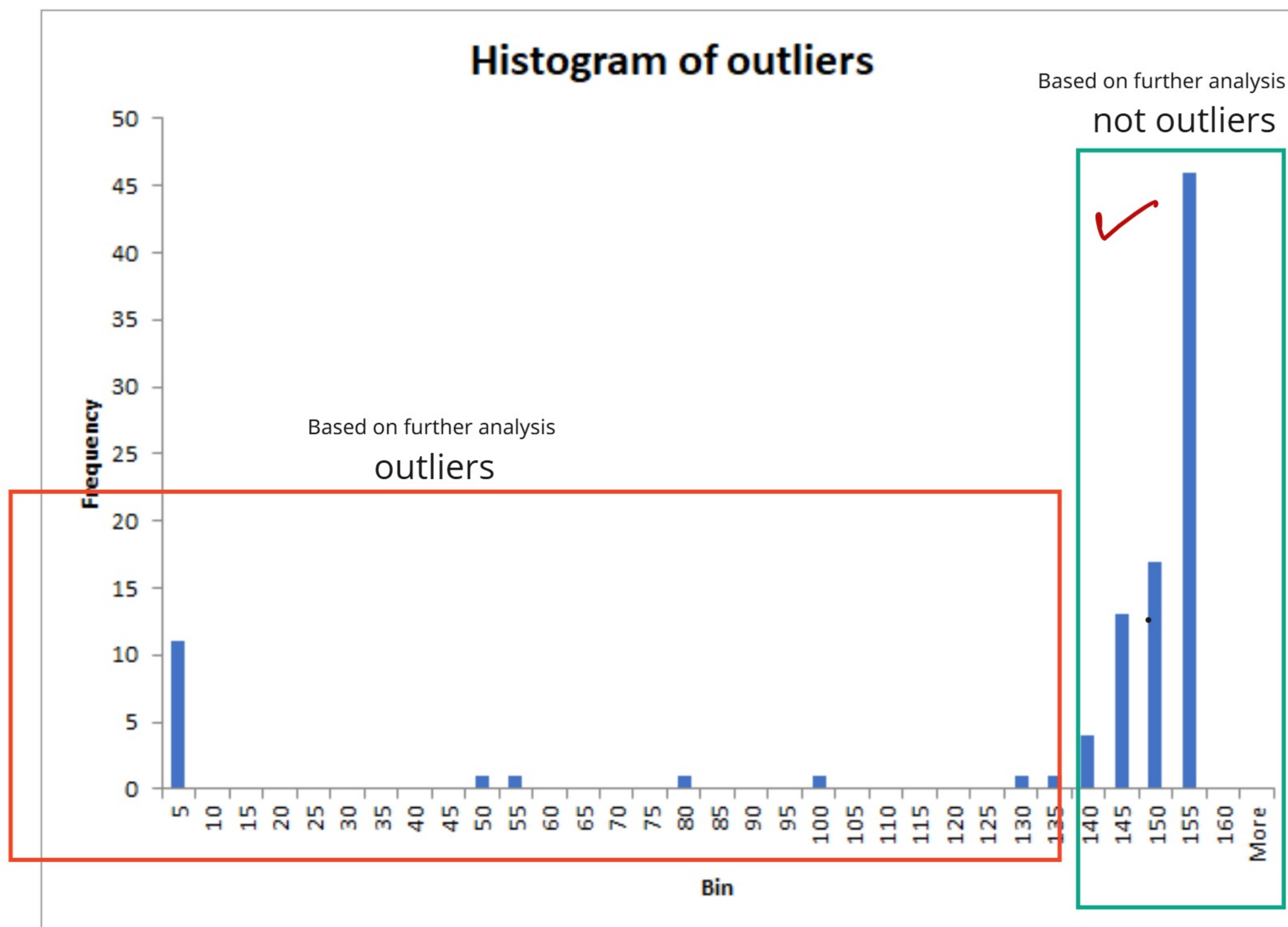


Line plot



Box or Whisker Plot

c4	
Mean	163.9284816
Standard Error	0.363181665
Median	166.1539939
Mode	0
Standard Deviation	14.9875705
Sample Variance	224.6272695
Kurtosis	94.71693965
Skewness	-9.197660397
Range	174.0257593
Minimum	0
Maximum	174.0257593
Sum	279170.2041
Count	1703
Quartiles	
Q1 (25th percentile)	163.2876459
Q2 (50th percentile)	166.1539939
Q3 (75th percentile)	168.4890902
IQR	5.201444298
LB	155.4854794
UB	176.2912566

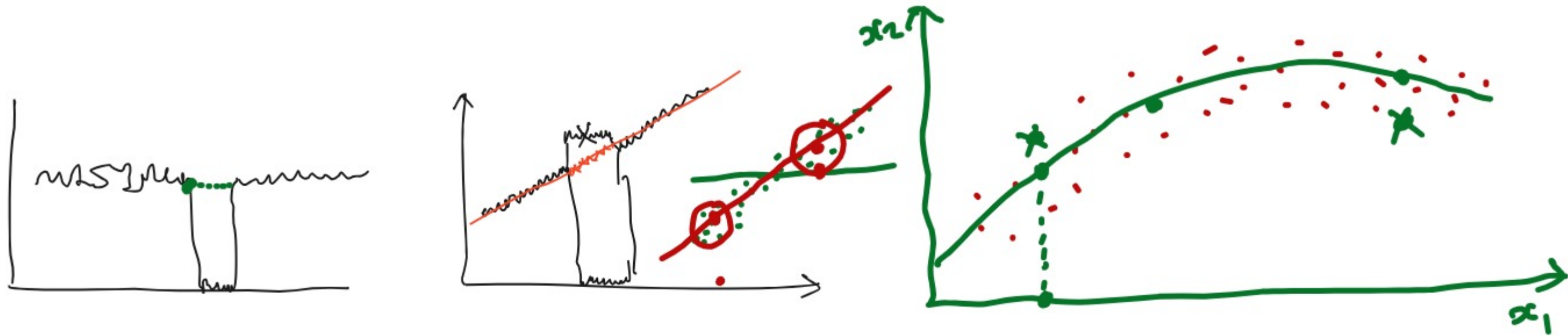


One way to further analyze the outliers and decide the cut-off bounds
(after applying the standard outlying detecting rule)

→ "DON'T BLINDLY APPLY
THE RULES"

{ Whenever we see an anomaly
(outliers, for example) it makes sense
to get into one level deeper analysis.

Strategies for replacing outliers



There are several strategies for replacing outliers during Exploratory Data Analysis (EDA). Here are some of the most accepted strategies:

- 1. **Winsorization**: This method replaces the extreme values with the nearest “good” data point, rather than truncating them completely
- 2. **Imputation**: Imputation involves replacing the outlier values with a reasonable estimate, such as the mean or median of the data set
- 3. **Trimming**: Trimming involves removing the outliers from the data set altogether
- 4. **Cap the data**: This method involves setting a cap on the maximum and minimum values of the data, so that extreme values are replaced with the maximum or minimum value
- 5. **Use robust estimation techniques**: Robust estimation techniques, such as certain kinds of regression that are less sensitive to outliers



Data trends are important while deciding outlier / missing data handling and data imputation methods to be used

When to Cap Outliers (Replace with Min/Max)

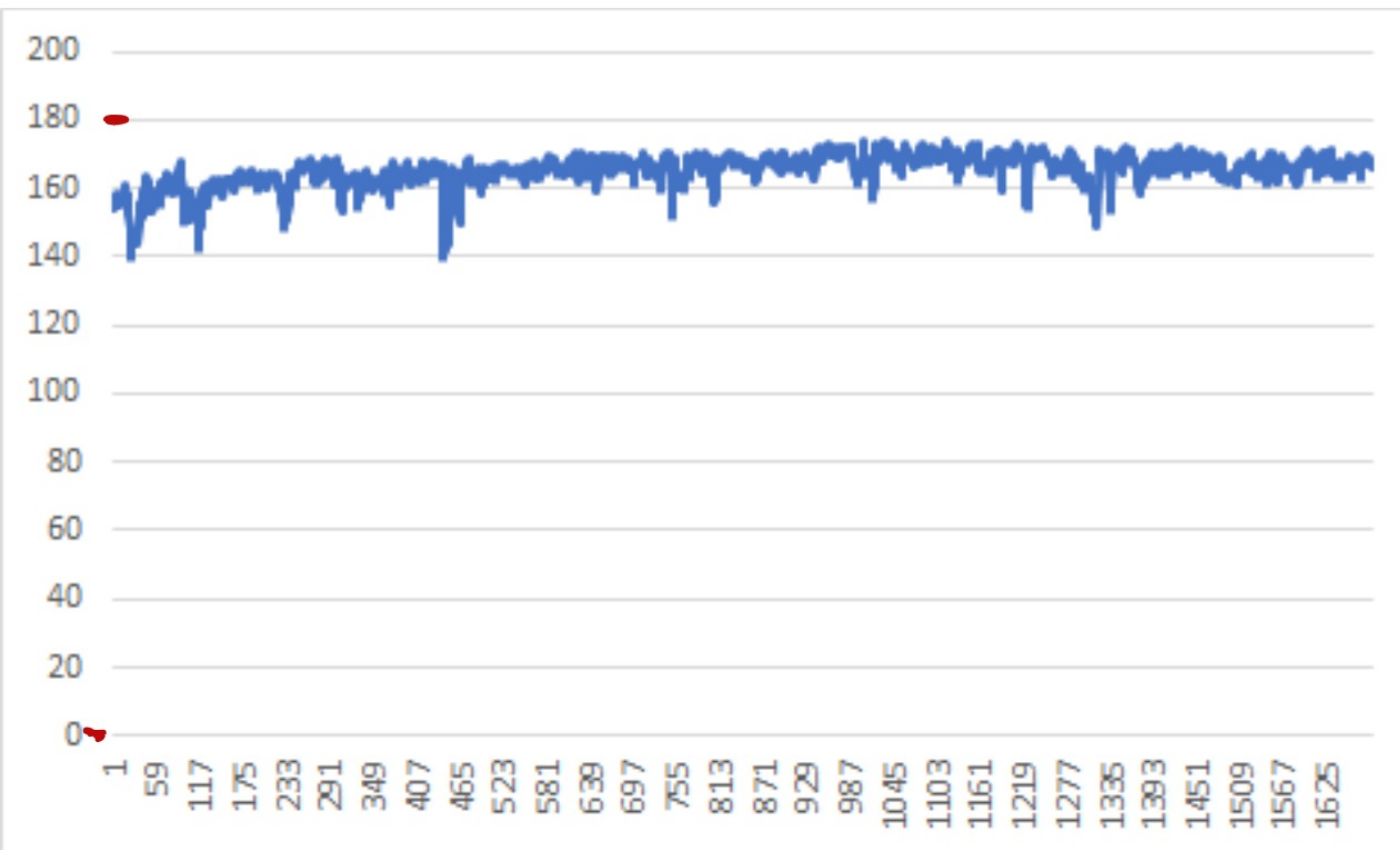
Condition	Should You Cap?	Reason	Example
Data entry errors or sensor issues	Yes	Outliers are not real, so capping corrects mistakes.	A temperature sensor records -500°C instead of 50°C due to a glitch.
Small dataset (outliers distort results)	Yes	One extreme value can skew calculations significantly.	A class of 10 students has test scores mostly around 70–80 , but one student scores 10 .
Predictive modeling (ML models sensitive to outliers)	Yes (for OLS, Ridge, Lasso)	Reduces overfitting and improves generalization.	In a linear regression model predicting house prices, one luxury home worth \$50M skews the model.
Business-defined limits (e.g., salary caps)	Yes	Capping aligns with real-world constraints.	A company caps bonuses at \$20,000 , so any value above is set to \$20,000 .
Extreme values create instability in analysis	Yes	Prevents distortions in means, variances, and model predictions.	A company's average revenue is \$10M , but one \$500M deal distorts financial forecasting.

When NOT to Cap Outliers (Keep the Raw Values)

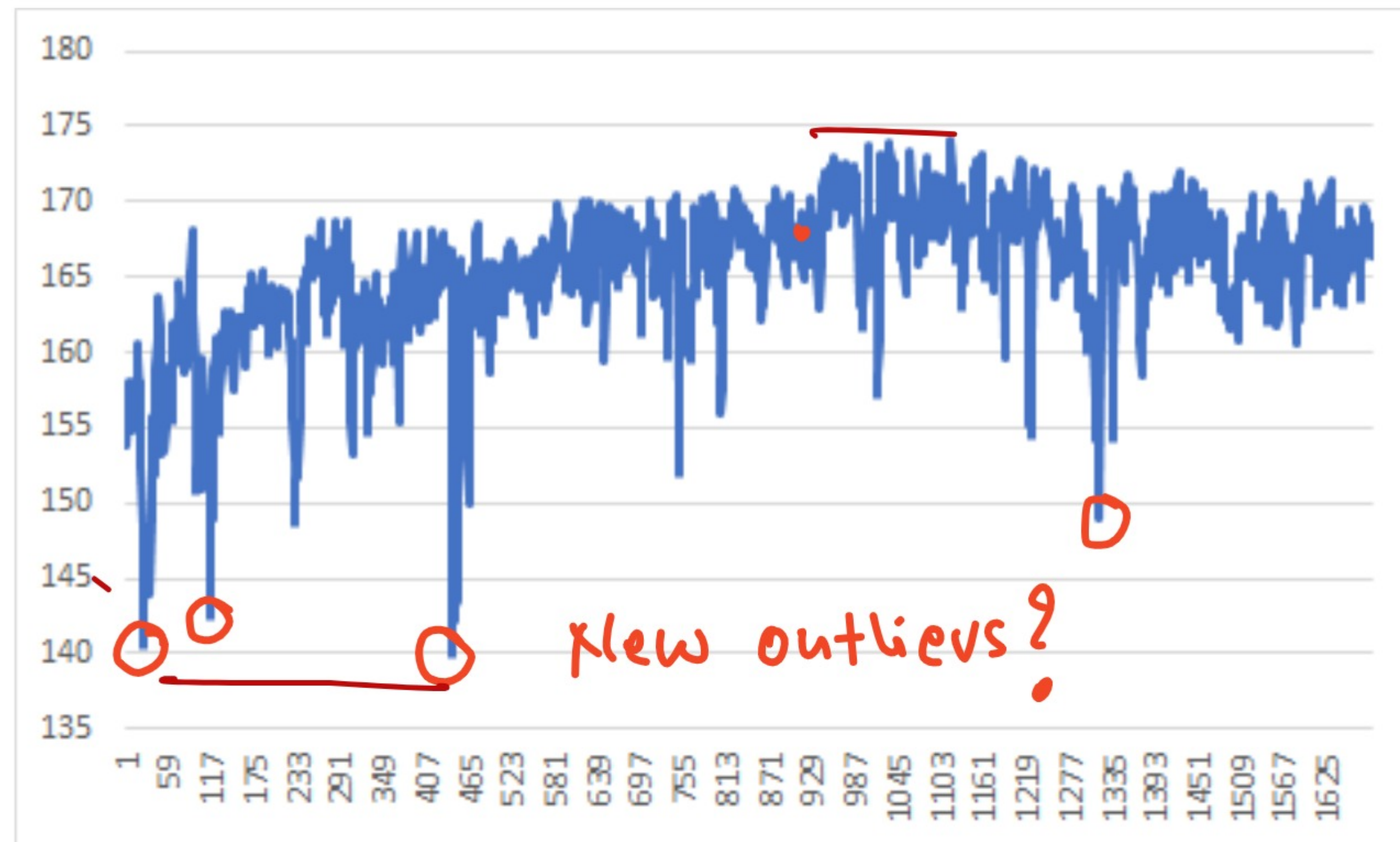
Condition	Should You Cap?	Reason	Example
Outliers carry important insights (e.g., fraud detection, stock market crashes)	No	Capping removes valuable information.	A credit card transaction of \$20,000 in a fraud detection system could be crucial evidence.
Large dataset (outliers have minimal impact)	No	A few extreme values won't distort overall results.	A dataset of 1 million sales records has a few very high transactions but remains stable overall.
Natural skewness in data (e.g., income, earthquake magnitudes)	No	Capping can misrepresent real-world distributions.	The richest 1% of a population earns far more than the rest; capping would mislead economic analysis.
Statistical models robust to outliers (e.g., Median, Quantile Regression, Random Forests, RANSAC)	No	These methods already handle extreme values well.	Random Forest regression can handle high-priced homes without needing to cap values.

Regression Methods and Outlier Sensitivity

Regression Method	Outlier Sensitivity	Notes
Linear Regression (OLS)	High	Highly sensitive due to the squared error term. Even one outlier can significantly affect the regression line.
Neural Networks	High	Flexible models prone to overfitting, easily influenced by outliers.
K-Nearest Neighbors (KNN)	High	Sensitive to outliers because it uses the distance metric to make estimations. Outliers can therefore heavily distort distances, and therefore predictions.
Tree-Based Methods (Decision Trees, Random Forests, Gradient Boosting)	Low-Moderate	Relatively robust due to their partitioning nature. Outliers tend to be isolated and have less influence on overall predictions. Can overfit outliers if trees grow too deep.
Robust Regression (Huber, RANSAC, Theil-Sen, Quantile)	Low	Designed specifically to be less sensitive to outliers. Various techniques handle outliers differently, from weighting them less to completely ignoring them.

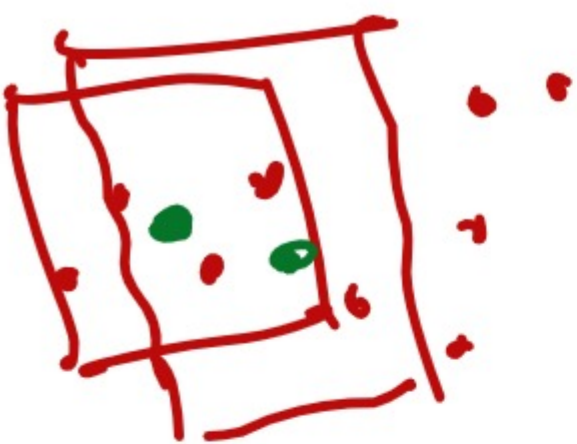
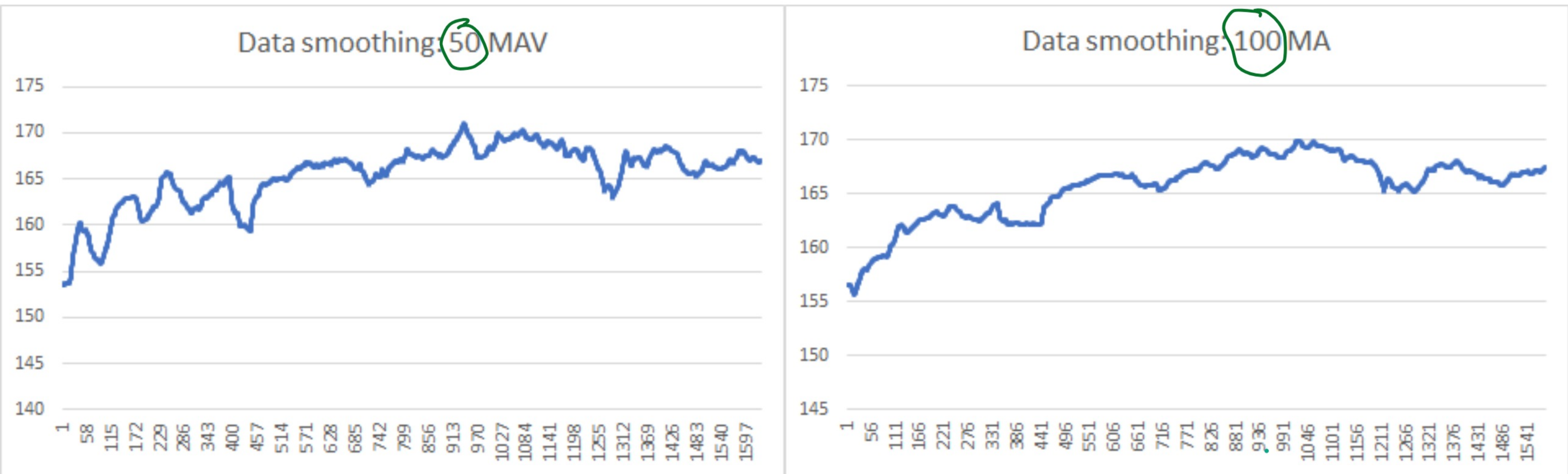


After 'fixing' the outliers



After re-scaling the display

If the data is based on 'time', Moving Averages can be created and advantageously used

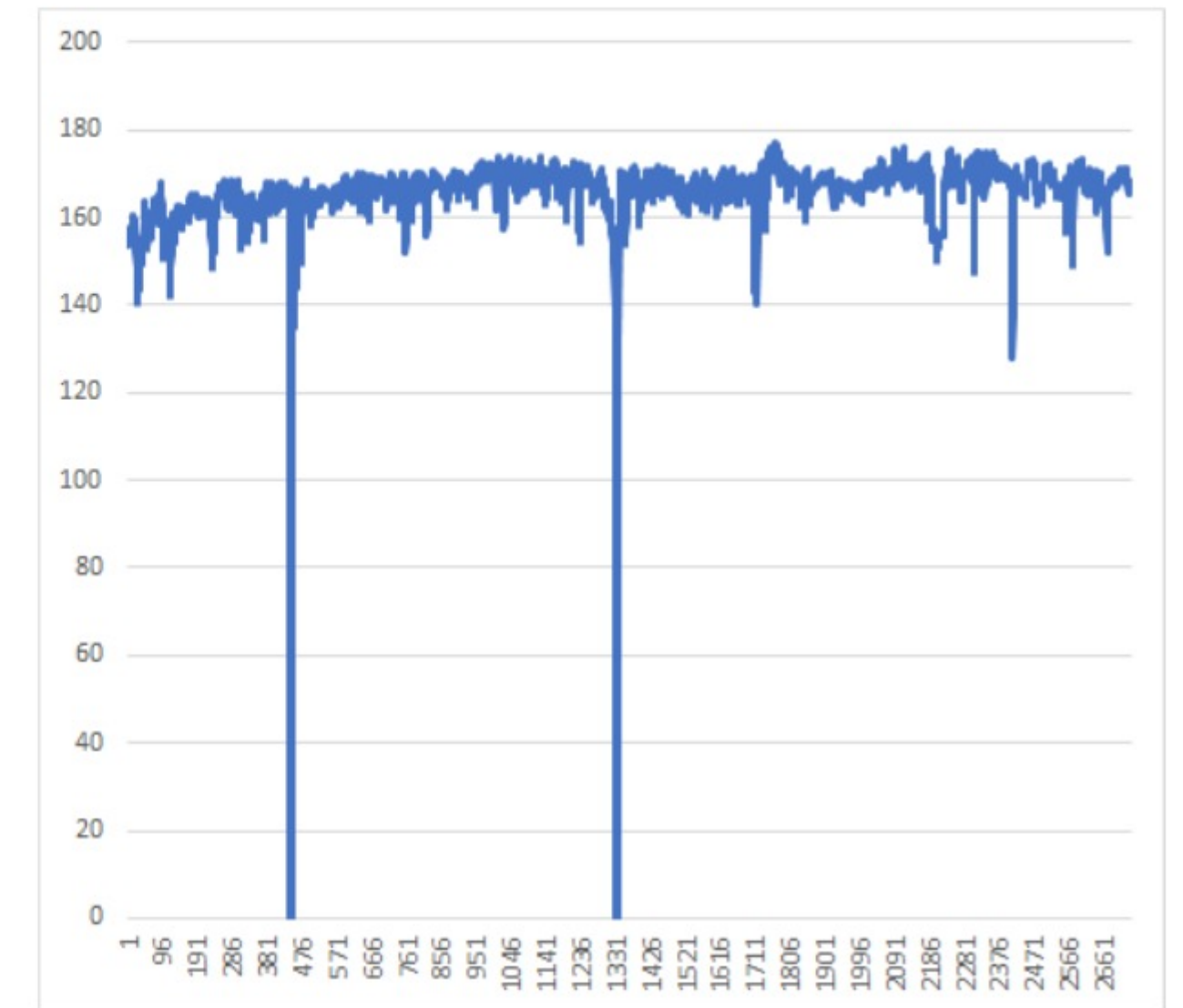
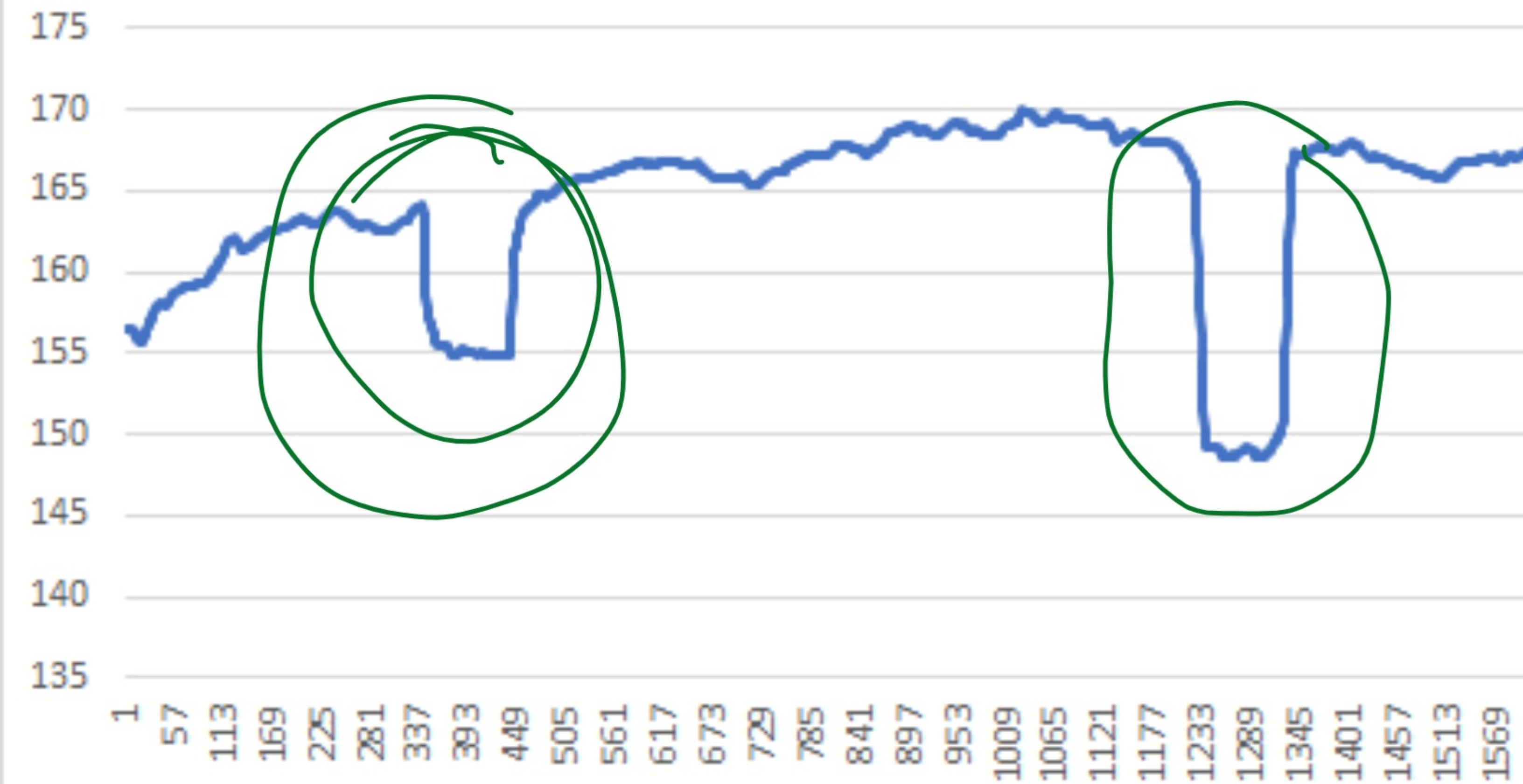


Moving averages can be used for:

- Filling up missing values
- Replacing outliers
- Eliminating noise
- Understanding data trends

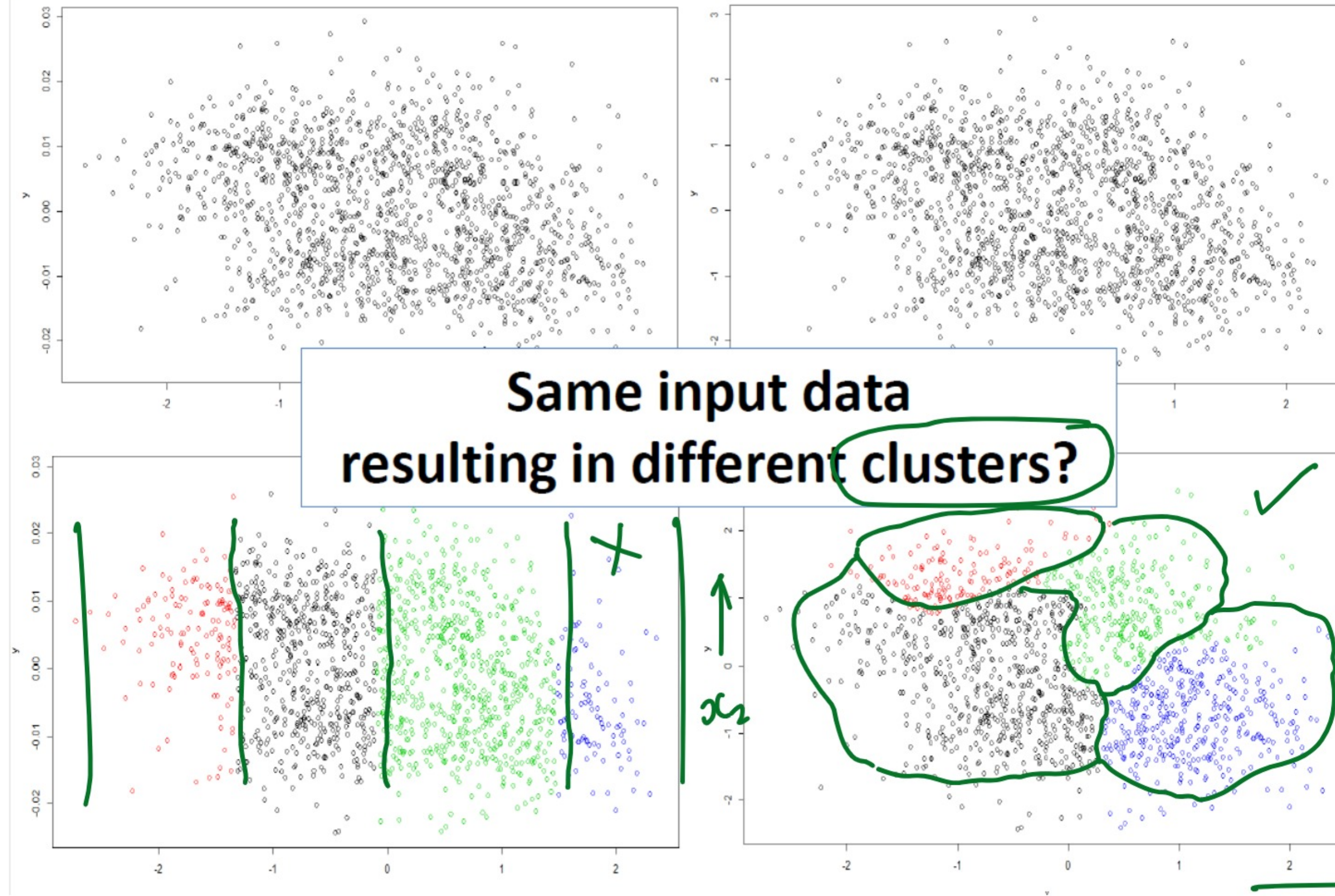
→
time

Effect of outliers on MA of raw data



Data Scaling: Impact of scale differences

Consider the situation shown below ...



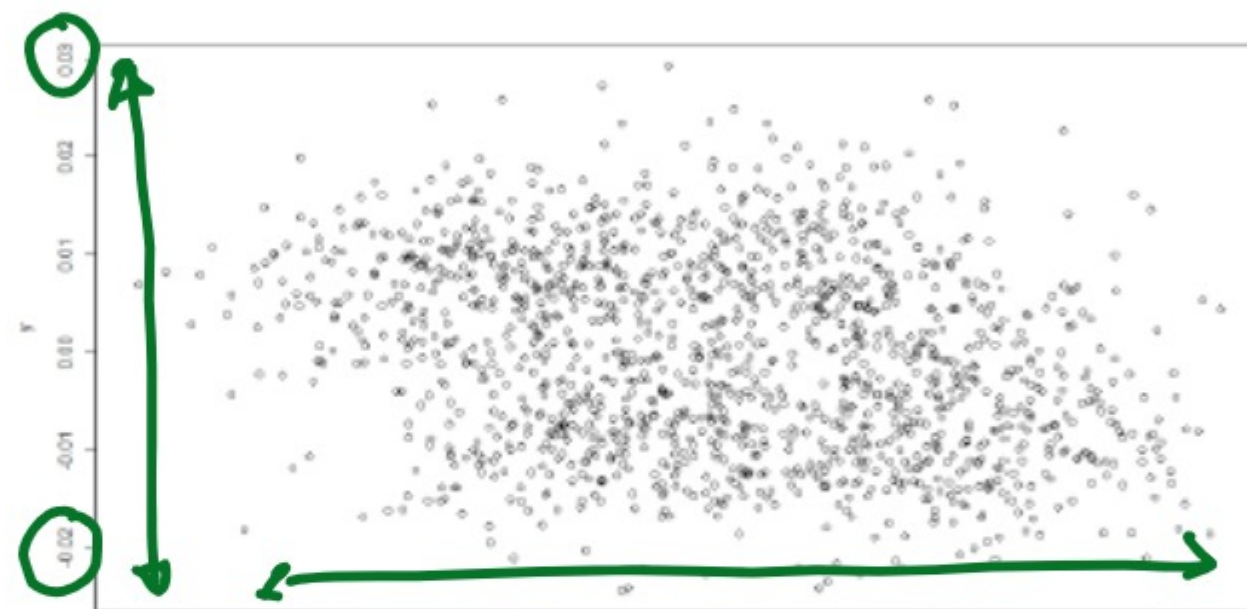
SCALING.

Max/min
Scaling
value

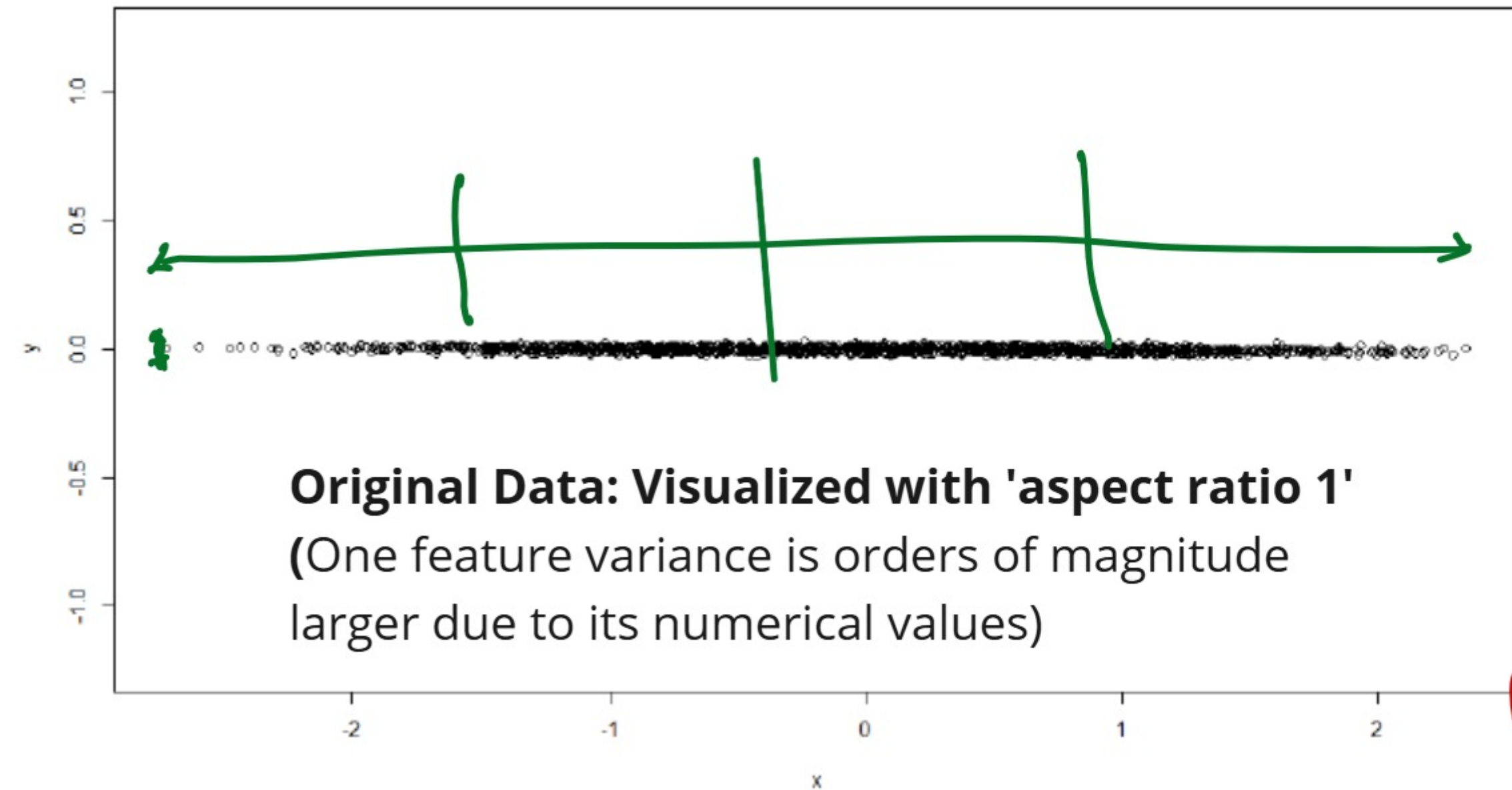
$$= \frac{(\bar{x}_i - x_{\min})}{(x_{\max} - x_{\min})}$$

Range

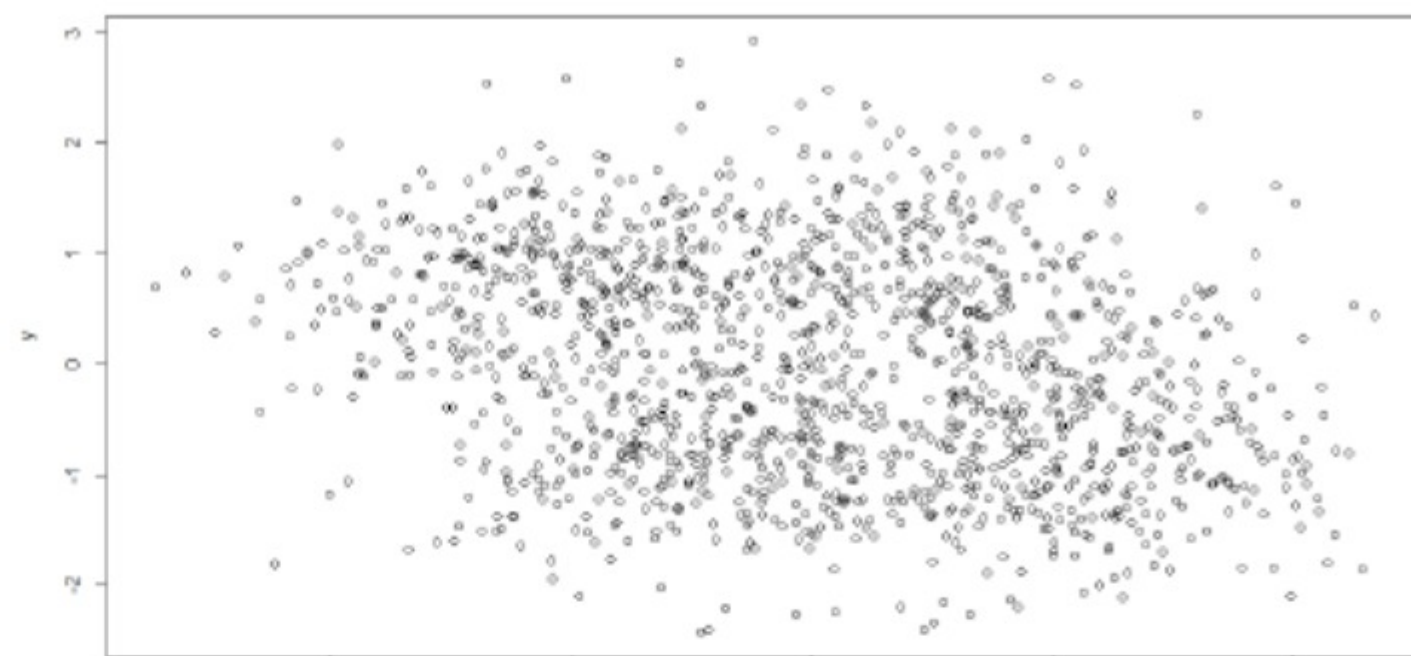
Data points re-drawn with aspect ratio 1



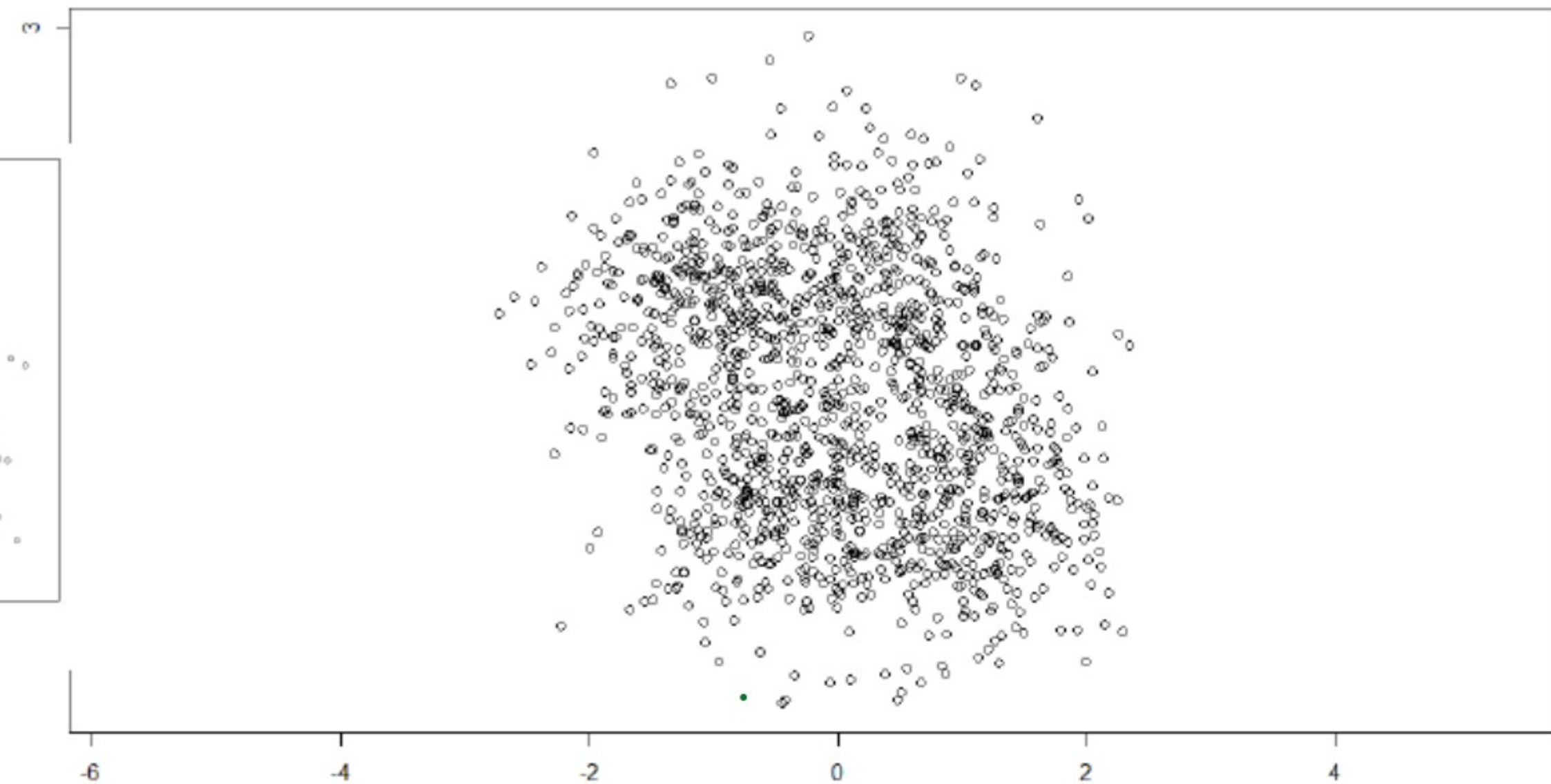
Original Data: Horizontal and vertical visualization scales are different



Original Data: Visualized with 'aspect ratio 1'
(One feature variance is orders of magnitude larger due to its numerical values)



Original Data



Data: after centering and scaling both features

Data Scaling and centering can be done using a few popular methods methods:

- **Normalization** →
- **Standardization** →

Normalization (Min-Max Scaling):

Equation:

Normal distribution

X_normalized = (X - X_min) / (X_max - X_min)

Where:

- X = the original value.
- X_min = the minimum value of the feature.
- X_max = the maximum value of the feature.
- X_normalized = the normalized value.

When Normalization is Beneficial

Condition	Why Normalize?	Example
Algorithms that rely on distance-based calculations	Many ML models (e.g., k-NN, SVM, Gradient Descent) are sensitive to feature scales.	k-NN uses Euclidean distance , which gets biased if one feature has much larger values than others.
Features have very different ranges	Prevents one large-scale feature from dominating the model.	Height (0–2m) vs. Salary (0–200,000); salary will dominate the model if not normalized.
Gradient-based optimization (e.g., neural networks, logistic regression)	Helps faster and more stable convergence.	Deep learning models train faster if inputs are scaled between [-1,1].
Principal Component Analysis (PCA)	PCA is variance-based, so large-scale features will dominate the principal components.	If one feature has values around 100,000 and another around 0.1 , PCA will give more importance to the large-scale one.

Common Normalization Methods:

- Min-Max Scaling (scales data to [0,1])
- Z-Score (Standardization) (scales to mean = 0, std = 1)
- Robust Scaling (uses medians for scaling, works well with outliers)

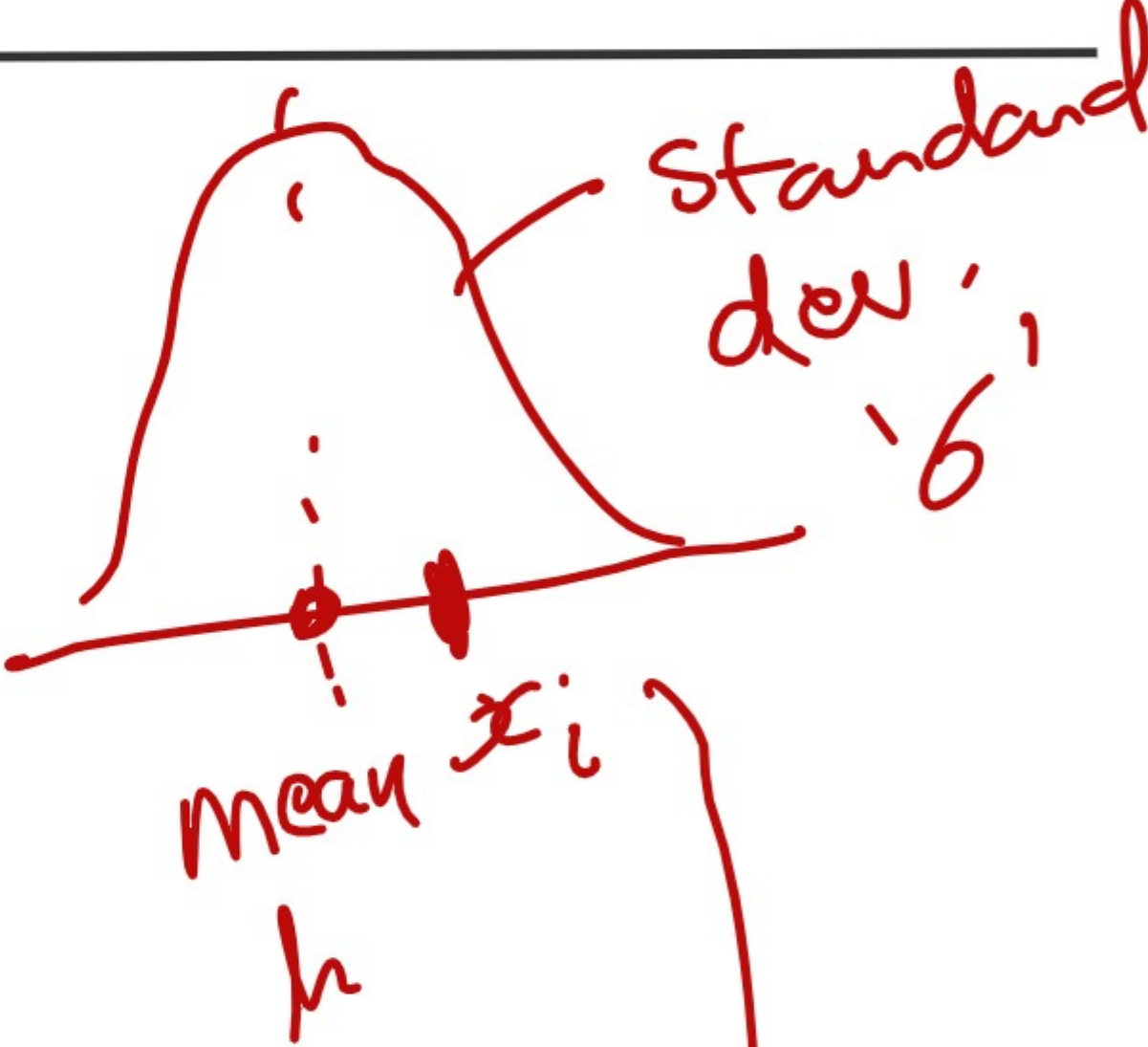
When Normalization is Unnecessary

Condition	Why Avoid Normalization?	Example
Tree-based models (e.g., Decision Trees, Random Forest, XGBoost)	Trees split data based on thresholds , so normalization does not affect performance.	A Random Forest predicting house prices works the same whether features are normalized or not .
Features have similar ranges already	If all features are in similar scales, normalization provides no real benefit .	If all features are between 0–100 , normalization won't improve results.
Normalization removes interpretability	Sometimes, raw feature values carry meaningful units that you don't want to lose.	Predicting house prices: a model that learns "higher square footage → higher price" is easier to interpret without scaling.
Normalization with outliers	Min-max scaling shrinks normal values but stretches extreme outliers, making them more dominant.	If income has an outlier (\$10M), most values will shrink near 0, losing meaningful variation. (Use RobustScaler instead!)

Standardization (Z-score Normalization):

Equation:

X_standardized = (X - μ) / σ



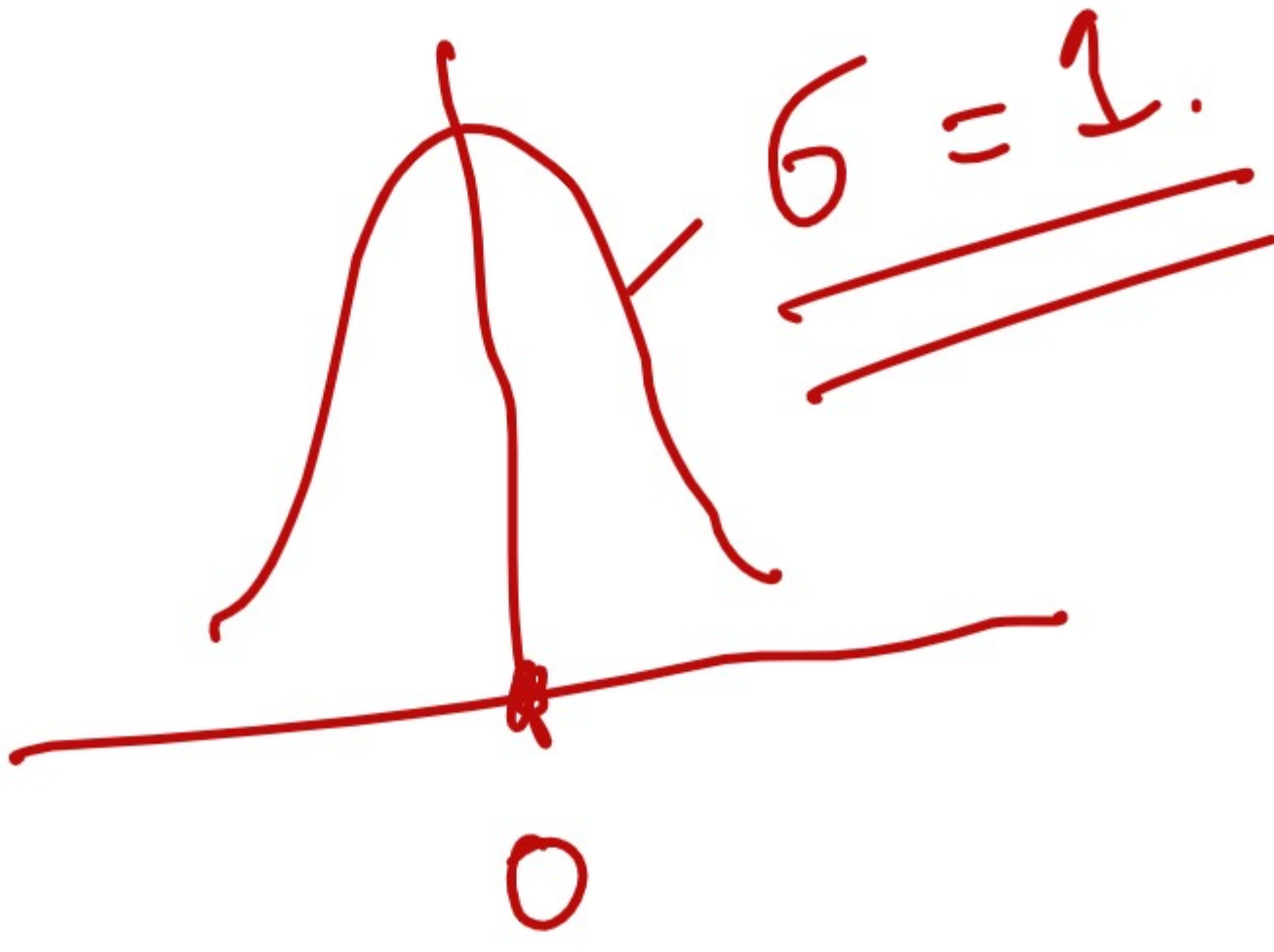
- X = the original value.
- μ = the mean of the feature.
- σ = the standard deviation of the feature.
- X_standardized = the standardized value.

x_i std = (x_i - μ) / σ

Algorithms That Benefit from Standardization

These algorithms work best when **features and residuals** follow a normal distribution.

Algorithm	Why Normality Matters?	Example of Impact
Linear Regression (OLS)	Assumes normally distributed residuals for accurate confidence intervals and p-values.	Skewed data may cause biased coefficient estimates .
Logistic Regression	Assumes linear relationship between independent variables and log-odds .	Strongly skewed predictors may lead to poor decision boundaries .
LDA (Linear Discriminant Analysis)	Assumes normally distributed features within each class .	If violated, LDA may perform worse than tree-based models.
Naïve Bayes (Gaussian NB)	Assumes Gaussian distribution of features .	Works poorly if features are skewed/non-Gaussian.
ANOVA (Analysis of Variance)	Assumes normally distributed residuals .	If violated, F-statistics become unreliable .
t-Tests (Statistical Hypothesis Testing)	Assumes normality to compare means.	Small sample sizes may give misleading results if data is skewed.



Algorithms That DO NOT Require Standardization

These algorithms **do not assume normality** and can handle **skewed or non-Gaussian data** well.

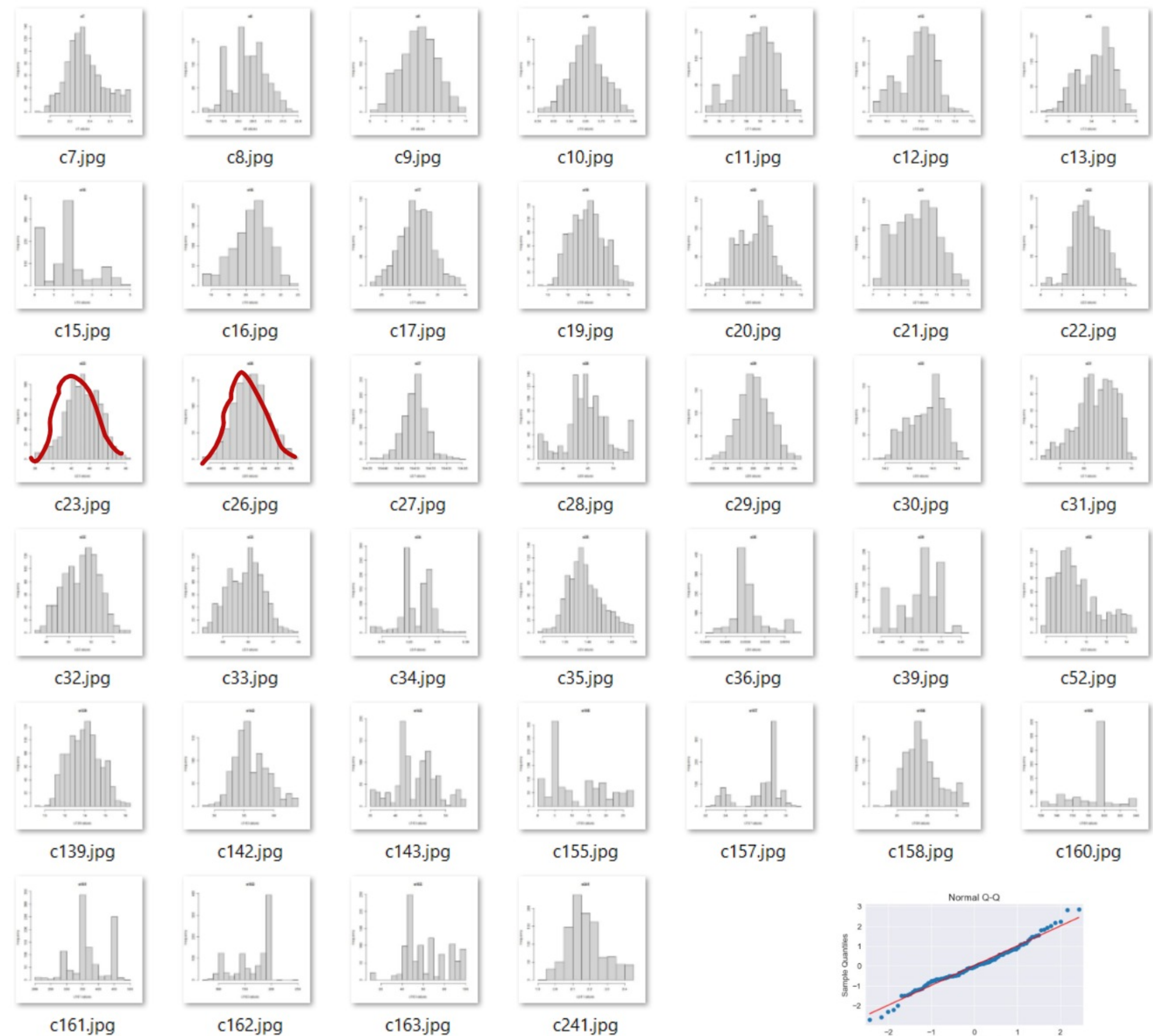
Algorithm	Why Normality is NOT Required?
Decision Trees (CART, Random Forest, XGBoost)	Splitting rules don't depend on distribution.
Support Vector Machines (SVM)	Uses kernel tricks , unaffected by feature distribution.
k-Nearest Neighbors (k-NN)	Distance-based, works better with normalization but not normality.
Neural Networks (MLP, Deep Learning)	Can learn complex, non-linear relationships without normality.
Gradient Boosting (GBM, LightGBM, CatBoost)	Works with skewed and non-Gaussian data.

Normalization:

- Use normalization when the distribution of the data is not Gaussian or when you have varying scales in your data.
- Normalize the data when the algorithm you are using does not make assumptions about the data distribution, such as k-nearest neighbors or artificial neural networks.
- Normalize the data when you want to bring all variables to the same range and preserve the shape and distribution of the data.

Standardization:

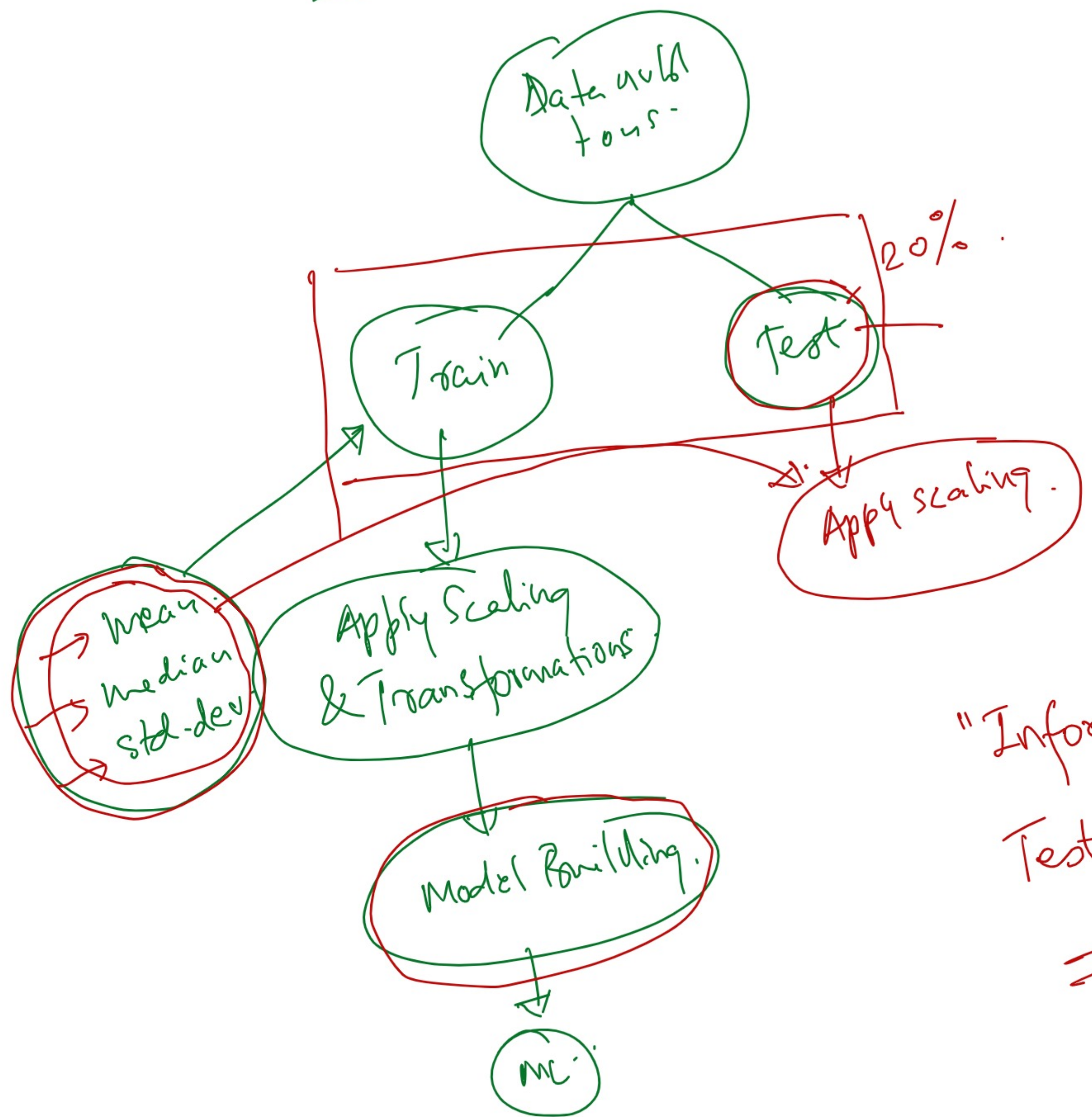
- Use standardization when the data follows a Gaussian distribution or when you see a bell-curve in your data.
- Standardize the data when you want to transform it to have a mean of 0 and a standard deviation of 1.
- Standardization is beneficial when dealing with unsupervised learning algorithms or when your dataset has extreme high or low values (outliers).



The distribution of data assumes significance in the context of **data scaling**

Scaling Method	Description	Use Cases / Scenarios
Standardization	Scales data to have mean 0 and standard deviation 1.	Suitable for algorithms assuming normal distribution, SVMs.
Normalization	Scales data to the range [0, 1] while preserving relative relationships.	Useful when features have varying ranges, distance-based algorithms.
Max Abs Scaler	Scales data by dividing by the maximum absolute value, preserves sparsity.	For sparse data, centered at zero, not sensitive to outliers.
Robust Scaler	Scales data using median and interquartile range to handle outliers.	When data contains outliers, preserving feature distributions.
Quantile Transformer	Transforms data to follow a uniform or normal distribution.	Mitigating impact of outliers, making data distribution more Gaussian-like.
Power Transformation	Applies power transformations to make data distributions more Gaussian-like.	For skewed data, making it more suitable for Gaussian-based models.
Unit Vector Scaling	Scales each feature by dividing by its magnitude, ensuring vectors have unit norm.	Useful for algorithms that rely on vector distances.
Log Transformation	Applies logarithmic transformation to data to compress large ranges.	Reducing impact of extremely large values, skewed data.
Mean Centering	Subtracts the mean of each feature from its values, resulting in mean-centered data.	When you want data centered around zero.

When to split available data into Train and Test sets?



"Information leakage"
Test data info leaked → Training data